



מבוא למדעי המחשב מ"ח' (234114 \ 234117)

מועד ב' קיץ תשפ"ה

מבחן מסכם, 26.11.25

2	3	4	1	1	
---	---	---	---	---	--

רשום/ה לקורס:

--	--	--	--	--	--	--	--	--	--

מספר סטודנט:

משך המבחן: שלוש שעות.

הנחיות כלליות:

- השימוש בכל חומר עזר אסור.
- בדקו שיש 10 עמודים (4 שאלות) במבחן, כולל עמוד זה.
- אלא אם כן נאמר אחרת בשאלות, אין להשתמש בפונקציות ספריה או בפונקציות שמומשו בכיתה, למעט פונקציות קלט/פלט והקצאת זיכרון (malloc, free). ניתן להשתמש בטיפוס bool המוגדר ב-stdbool.h.
- אין להשתמש במשתנים סטטיים וגלובליים אלא אם נדרשתם לכך מפורשות.
- הקפידו על סגנון כתיבה כזה שקורא התוכנית ידע לפענח את רעיונותיכם:
 - התוכנית יכולה להכיל תיעוד קל להבנה.
 - על התוכנית להיות כתובה באופן מסודר ומדורג.
 - יש לתת שמות פונקציות ושמות משתנים משמעותיים.
 - ערכים קבועים יש להגדיר בעזרת define.
 - ניתן להוסיף פונקציות עזר כרצונכם.
- נוהל "לא יודע": אם תורידו את ההערה מהקריאה לפונקציה printIDontKnow() על שאלה שבה אתם נדרשים לקודד, תקבלו 20% מהניקוד. דבר זה מומלץ אם אתם יודעים שאתם לא יודעים את התשובה.
- שימו לב שקיבלתם את קבצי ה C ובהם שלד של התוכנית. עליכם רק להשלים את המימוש של הפונקציה הנדרשת. אין לשנות את ה main.
- לכל שאלה קיבלתם מספר טסטים. מומלץ להוסיף טסטים שלכם.

המשך ההנחיות בעמוד הבא.



הנחיות הגשה: חשוב לקרוא בעיון!

- (1) לפני תחילת המבחן, יש למלא במודל הצהרה על טוהר הבחינות. סיסמה להצהרה במודל נמצאת בעמוד 3. ניתן למלא את ההצהרה רק בחצי השעה הראשונה.
- (2) הגשת כל שאלה במטלה נפרדת במערכת Gradescope רק לאחר ההצהרה על טוהר הבחינות, ולא בחצי שעה הראשונה.
- (3) יש למלא הצהרה על סיום הבחינה לאחר הגשת המטלה האחרונה שפתרתם ולפני היציאה מהכיתה.
- (4) במהלך המבחן מותר לעבוד אך ורק בסביבת CLion.
- (5) אין להשתמש בשום כלי בינה מלאכותית יוצרת כגון: CoPilot, ChatGPT וכדומה. שימוש בכלים אלו יגרור הליכים משמעותיים.
- (6) הגשה לא לפי השלבים שפורטו לעיל, ובפרט: אי הצהרה על טוהר הבחינות/ הצהרה על סיום הבחינה, או הגשת מטלה לאחר הצהרה על סיום הבחינה, **תוביל לציון 0** והליכים משמעותיים.

נוסחאות שימושיות:

$$1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} = \Theta(\log n) \quad \log 1 + \log 2 + \dots + \log n = \Theta(n \log n)$$

$$1^k + 2^k + 3^k + \dots + n^k = \Theta(n^{k+1}) \quad 1 + k + k^2 + k^3 + \dots + k^n = \Theta(k^n), \quad k > 1$$

$$S_n = \frac{a_1}{1-q} \text{ : אינסופית, } S_n = a_1 * \frac{q^n - 1}{q - 1} \text{ : סכום סדרה הנדסית}$$

$$S_n = n * \frac{a_1 + a_n}{2} \text{ : סכום סדרה חשבונית}$$

$$\log_a(m \cdot n) = \log_a(m) + \log_a(n)$$

$$\log_a\left(\frac{m}{n}\right) = \log_a(m) - \log_a(n)$$

$$\log_a(m^n) = n \log_a(m)$$

$$n^{\log_n m} = m$$

234114/7 צוות הקורס

מרצים: גב' יעל ארז; מתרגלת אחראית: זהר נגל.



סיסמה להצהרה על טוהר הבחינות:

Milk&Honey





שאלה 1: [25 נקודות]

ענו על השאלה במטלה המתאימה ב-Gradescope. לכל סעיף עם קטע קוד יופיעו שני תתי-סעיפים, אחד לסיבוכיות זמן ואחד לסיבוכיות מקום. בכל תת-סעיף עליכם לבחור אפשרות אחת ולסמן אותה. בכל הסעיפים יש להניח כי סיבוכיות הזמן והמקום של פונקציות הספרייה המוכרות *printf*, *free* היא $\Theta(1)$. בנוסף יש להניח כי סיבוכיות הזמן של *malloc(n)* היא $\Theta(1)$ וסיבוכיות מקום נוסף היא $\Theta(n)$.

א. חשבו את סיבוכיות הזמן והמקום של הפונקציה *f1* המוגדרת בקטע הקוד הבא, כפונקציה של *n*. אין צורך לפרט את שיקוליכם. חובה לפשט את הביטוי ככל שניתן.

```
int f1(int** arr, int n, int value) {
    for (int row = 0; row < n; row++) {
        int col = 0;
        while (col < n && arr[row][col] < value) col++;
        int* b = malloc((n + 1) * sizeof(int));
        if (!b) return n;
        for (int j = 0; j < col; j++) b[j] = arr[row][j];
        b[col] = value;
        for (int j = col+1; j <= n; j++) b[j] = arr[row][j-1];
        arr[row] = b;
    }
    return n + 1;
}
```

ב. חשבו את סיבוכיות הזמן והמקום של הפונקציה *f2* המוגדרת בקטע הקוד הבא, כפונקציה של *n*. אין צורך לפרט את שיקוליכם. חובה לפשט את הביטוי ככל שניתן.

```
void g2(int n, int m) {
    if (n <= m) return;
    int *arr = malloc(sizeof(int) * m);
    if (!arr) return;
    free(arr);
    g2(n / 2, m * 2);
    g2(n / 2, m * 2);
}
void f2(int n) {
    g2(n, 1);
}
```



- ג. בקובץ `finalB_q1.c` תוכנית שבה קולטים שמות סטודנטים מהמשתמש ושומרים אותם במטריצה `names`.
- ד. הפונקציה:

```
bool read_names(char*** names, int* n);
```

- ה. קוראת את מספר הסטודנטים, ושמות הסטודנטים מהמשתמש ושומרת אותם במערך `names`. מובטח כי השם הארוך ביותר לא יהיה יותר מ `MAX_LEN` תווים.
- הפונקציה:

```
void free_names(char** names, int n);
```

- משחררת את הזכרון שהוקצה באופן דינמי.
- בקובץ יש גם פונקציית `main` אשר קוראת לפונקציית `read_names`, מדפיסה את השמות, וקוראת לפונקציית `free_names`.

בקובץ יש טעויות. בפתרון עליכם להגיש את התוכנית המתוקנת.

לרשותכם הכנו בגריידיסקופ טסט שנכשל עם התוכנית הנתונה.

שימו לב:

- ב `main` אין טעויות ואסור לשנות אותו.
- אין לשנות את חתימות הפונקציות.
- זה שהטסט עובר לא אומר שתיקנתם את כל הטעויות.
- כל הקצאה דינמית יש לוודא שהצליחה.
- יש לשחרר כל זכרון שהוקצה דינמית.
- פונקציית המקבלת מצביע, תמיד צריכה לבדוק שהוא שונה מ `NULL` לפני שתשתמש בו.



שאלה 2: [25 נקודות]

עליכם לממש את הפונקציה הבאה:

```
int exam_q2(int arr[], int n, char c, int x);
```

הפונקציה מקבלת מערך arr באורך n של מספרים שלמים ללא חזרות, תו סידור c ומספר x . עליכם למצוא האם התו x נמצא במערך. אם כן, יש להחזיר את האינדקס שלו, אחרת יש להחזיר -1.

נתון שהמערך הנתון מסודר לפי אחד מהכללים הבאים, כתלות בתו c .

התו c	סידור המערך
'u'	המערך ממוין בסדר עולה
'd'	המערך ממוין בסדר יורד

עליכם להגדיר פונקציית עזר עם החתימה הבאה:

```
bool compare(int a, int b, char c);
```

לפונקציה $exam_q2$ אסור להשתמש במשתנה c פרט להעברתו כפרמטר ל- $compare$. אסור להגדיר פונקציות עזר נוספות.

דוגמאות:

תוכן המערך	c	x	ערך החזרה
1, 2, 3, 4, 5	'u'	3	2
10, 5, 2, -1, -3	'd'	7	-1
12, 0, -2	'd'	0	1

הערות:

- נתונה לכם תוכנית אשר קוראת מהמשתמש את אורך המערך, ולאחר מכן את תוכנו ואת התווים c ו- x .
- ניתן להניח שהקלט תקין, בפרט arr לא מכיל את אותו איבר פעמיים, c הוא אחת מהאפשרויות לעיל ו- arr מסודר בהתאם אליו.

דרישות סיבוכיות:

- סיבוכיות זמן עבור הפונקציה $compare$: $O(1)$.
- סיבוכיות זמן כוללת: $O(\log(n))$.
- סיבוכיות מקום נוסף: $O(1)$.



שאלה 3 [25 נקודות]

עליכם לממש את הפונקציה הבאה: $\text{char} * \text{exam_q3}(\text{char} * s1, \text{char} * s2);$

הפונקציה מקבלת 2 מחרוזות שמייצגות מספרים שלמים אי שליליים. על הפונקציה להחזיר את הסכום של מספרים אלו. על הפונקציה להחזיר את התוצאה במחרוזת שתוקצה דינאמית.

דוגמאות

המחרוזת s1	המחרוזת s2	מחרוזת התוצאה
"1"	"1"	"2"
"998"	"2"	"1000"
"5"	"0"	"5"
"123456789012"	"1"	"123456789013"

הערות:

- נתונה לכם תוכנית אשר קוראת מהמשתמש את המחרוזות s1 ו-s2 (אורך ותוכן), קוראת לפונקציה ומדפיסה את המחרוזת החדשה (ומשחררת אותה).
- ניתן להניח ש-s1 ו-s2 מחרוזות חוקיות שמכילות רק ספרות.
- המחרוזות לא יכילו אפסים מובילים (אבל הן יכולות להיות "0").
- לא ניתן להניח שהמספרים המתקבלים יכולים להיות מיוצגים כ-*int* ואפילו לא כ-*long long*.

דרישות סיבוכיות:

- סיבוכיות זמן: $O(n)$ כאשר n הוא אורך המחרוזת הארוכה יותר.
- סיבוכיות מקום: $O(n)$.



שאלה 4: [25 נקודות]

יוסי מנסה לתכנן טיול מסביב לעולם דרך מדינות שנמספר $0, 1, 2, 3, \dots, N-1$. יוסי רוצה לעבור בכל מדינה בדיוק פעם אחת, לא פחות ולא יותר. לשם כך הוא נעזר במטריצת מעברים בין המדינות בשם `routes` בגודל $N \times N$ המייצגת את כל המסלולים הקיימים. אם קיים מעבר ממדינה i למדינה j אזי `routes[i][j]` יהיה מחיר המעבר. אם לא קיים מעבר, `routes[i][j]` יהיה 0.

שימו לב שהמטריצה לא חייבת להיות סימטרית, כלומר ייתכן שקיים מעבר מ i ל j אך לא קיים מעבר מ j ל i , או שהמחיר שונה.

עליכם לממש את הפונקציה הבאה: `int exam_q4(int routes[N][N]);` אשר מחזירה את מחיר המסלול הזול ביותר שמקיים את כל הדרישות, ו 0 אם לא קיים מסלול כזה.

למשל, עבור $N=3$ (3 מדינות) ומטריצת המעברים הבאה:

	$j = 0$	$j = 1$	$j = 2$
$i = 0$	0	200	300
$i = 1$	50	0	25
$i = 2$	150	0	0

יוסי יכול לטייל במסלול הבא: לצאת ממדינה 1 למדינה 2 (עולה 25) ואז ממדינה 2 למדינה 0 (עולה 300). סה"כ יוסי ישלם 325.

מסלול אחר יכול להיות ממדינה 0 למדינה 1 (עולה 200) וממדינה 1 למדינה 2 (עולה 25). סה"כ המחיר 225, יקר יותר מהאפשרות הראשונה. קיימים עוד מסלולים, כולם יקרים יותר.

עבור 3 מדינות ומטריצת המעברים הבאה:

	$j = 0$	$j = 1$	$j = 2$
$i = 0$	0	200	0
$i = 1$	0	0	0
$i = 2$	0	100	0

אם נתחיל ממדינה 0 או מדינה 2, נוכל לעבור רק למדינה 1 וממנה אין מעבר לאף מדינה. אם נתחיל ממדינה 1 לא נוכל לעבור לאף מדינה. לכן לא קיים מסלול העובר בכל המדינות והפונקציה תחזיר 0.

המשך השאלה בעמוד הבא



הערות:

- נתונה לכם תוכנית אשר קוראת מהשתמש את המטריצה routes, קוראת לפונקציה ומדפיסה למסך את המחר.
- יש להשתמש בשיטת backtracking כפי שנלמדה בכיתה.
- בשאלה זו אין דרישות סיבוכיות, אולם כמקובל ב-backtracking יש לוודא שלא מתבצעות קריאות רקורסיביות מיותרות עם פתרונות שאינם חוקיים.
- ניתן ומומלץ להשתמש בפונקציות עזר (ויש לממש את כולן).
- ניתן להניח כי המטריצה מכילה ערכים אי שליליים בלבד, וכי איברי האלכסון כולם 0.

בהצלחה!